

Aman Pandey Sanjeev Kumar

☎ +1 (765)-543-8366 ✉ amanpandey.sanjeevk@gmail.com [in LinkedIn](#) [GitHub](#) [Website](#)

Education

PURDUE UNIVERSITY

Master of Science in Electrical and Computer Engineering

Specialization: Artificial Intelligence & Machine Learning

West Lafayette, Indiana

May 2026

Technical Skills

Programming & GPU Kernels: Python, C++, CUDA, Triton, CuTe DSL, CUTLASS

Frameworks & Compilers: PyTorch, `torch.compile`, TorchDynamo, TorchInductor, FX, ONNX, MLIR, XLA, TVM

LLM Inference & Serving: vLLM, SGLang, TensorRT-LLM, Continuous Batching, KV Caching, Prefix Caching, Prefill/Decode, TTFT, TPOT/ITL, End-to-End Latency, Throughput, Goodput/SLO Analysis, Tensor/Pipeline/Expert Parallelism

Profiling & Benchmarking: PyTorch Profiler, Nsight Systems, Nsight Compute, Perfetto, Benchmark Harness Development, Warmup/Steady-State Timing, Correctness Validation

Quantization & Precision: PTQ, AWQ, SmoothQuant, per-tensor, per-channel, and group/block scaling; symmetric versus asymmetric integer quantization, INT8/INT4, FP16/BF16, FP8, NVFP4, NF4,

Systems & Deployment: Linux, Docker, Kubernetes, Slurm, Git, Bash

Projects

GPU Kernel Optimization for LLM Inference | [GitHub](#)

- ★ Built **FP32 RMSNorm** and **GEMV/small-M GEMM** kernels in **CUDA C++** and **Triton** on RTX 5050.
- ★ Fused residual add, RMS reduction, and scaling: **7 eager PyTorch launches to 1 kernel** at 256×4096 .
- ★ Row tiling and split-K accelerated two $M=16$ Triton projections $15.3\text{--}16.3\times$ over earlier implementations (CUDA Graph medians).
- ★ Passed **80/80 backend-shape checks** against **PyTorch**, including tails, KN/NK weight layouts, and CUDA Graph replay.
- ★ Benchmarked against **PyTorch** and **cuBLAS/cuBLASLt**; profiled with **PyTorch Profiler** and **Nsight Systems/Compute**.

nano-vLLM: Single-GPU LLM Inference Runtime | [GitHub](#)

- ★ Built **token-budget continuous batching** and chunked prefill with OpenAI-style serving and **TTFT/TPOT/KV telemetry**.
- ★ Implemented **paged KV tables**, refcounts, and prefix caching with **parent-hash chaining**; audited **1,500 seeded operations**.
- ★ Measured **$3.84\times$ output throughput ($58 \rightarrow 223$ tok/s)** vs. single-sequence scheduling on 12 mixed-length Qwen3-0.6B requests.
- ★ Warm 256-token prefix reuse avoided **3,072 prefill tokens** and cut median **TTFT $139 \rightarrow 34$ ms**; all 12 greedy outputs matched.
- ★ NF4 cut loaded CUDA allocation **$1.11 \rightarrow 0.50$ GiB** vs. BF16; evaluated INT8 loading and INT8/packed-INT4 **storage prototypes**.

PyTorch Compiler & LLM Inference Performance Analysis | [GitHub](#)

- ★ Built a **`torch.compile` suite on RTX 5050** with FX capture, correctness gates, isolated caches, and fresh-process profiling.
- ★ Rewrote tensor-dependent control flow with **`torch.cond`**, reducing three FX regions to **one full graph** across both tested branches.
- ★ Measured **$2.07\text{--}2.12\times$ FP32 pointwise speedup**; profiled **$13.1\text{--}16.2\%$ higher latency** in a compiled 4.7M-parameter MLP.
- ★ Profiled **Qwen2.5-0.5B FP32, CUDA Graphs disabled**: prefill kernels **$1,385 \rightarrow 482$** (44 tokens); decode **$8,968 \rightarrow 3,280$** (8 steps).
- ★ Traced **compiler-generated fusion** and attention changes; **8-step decode $97.9 \rightarrow 66.8$ ms**, max logit error 6.1×10^{-5} .

Triton FlashAttention-2: Forward Attention | [GitHub](#)

- ★ Implemented **FA2-style forward attention** in **Triton** with online softmax, avoiding the $N \times N$ **score matrix**.
- ★ Supported **FP16 MHA/GQA** at $D = \{32, 64, 128\}$; added **causal tile pruning** and masking for partial sequence tiles.
- ★ Validated **60/60 CUDA forward cases** vs. PyTorch SDPA across $B = \{1, 2, 4\}$; max absolute output error 2.44×10^{-4} .
- ★ Measured **$1.15\times/1.13\times$ speedups** vs. eager/`torch.compile` SDPA (default), causal $B = 1, H = 16, N = 511, D = 64$ (RTX 5050).
- ★ Profiled with **Nsight Systems** and **Nsight Compute**; measured **255 registers/thread** and **8.3% occupancy** at $D = 128$.

MobileCLIP2 Optimization for Qualcomm QNN (LPCVC 2026) | [GitHub](#)

- ★ Deployed MobileCLIP2-S2 as **FP16 QNN DLCs** on Qualcomm AI Hub's S22 Family target: image/text **P90 $14.51/3.99$ ms** (18.50 ms sum vs. 35 ms) with **794/794 profiled ops on NPU**.
- ★ Built a static **PyTorch $\rightarrow$ ONNX opset-16 $\rightarrow$ QNN** pipeline with FastViT reparameterization and normalization/SDPA/EOS rewrites; fixed preprocessing to raise fractional R@10 **$0.043 \rightarrow 0.897$** .
- ★ Validated all **56 image/211 text** device outputs at unchanged R@10 **0.897470** and cosine **$0.999863/0.999998$** ; rejected calibrated W8A16 QDQ at image cosine **0.811** and isolated an S22/v69 DLC-composition failure.

Experience

Teaching Assistant, IE 332: Computing in Industrial Engineering

Purdue University

Aug 2023 – Dec 2025

West Lafayette, Indiana

- ★ Led lab sessions for 75 students on Machine Learning algorithms, data pipelines, and supervised/unsupervised learning.
- ★ Taught relational database design, ERDs, SQL, and backend web development (HTML/CSS/JS/PHP/MySQL).
- ★ Guided student projects integrating ML with ERP, CRM, and SCM systems.

Data Science Researcher

Cisco X Purdue Data Mine

Jan 2024 – May 2024

West Lafayette, Indiana

- ★ Built forecast selection & ensembling algorithms for 10k+ Cisco products, improving cost-weighted prediction accuracy by **3%**.
- ★ Clustered PLIDs using **Gaussian Mixture Models** to refine forecast segmentation.
- ★ Applied Bayesian MCMC, MLE, and advanced calibration methods to optimize ensemble weighting.